Phase 0 Foundation Files

src/lib/types.ts

```ts
export type RightsLevel =
  | "educational-ok"
  | "permission-granted"
  | "unknown"
  | "restricted"
  | "public-domain";

export type CauseTag =
  | "Racial Justice"
  | "Women's Rights"
  | "LGBTQ+ Rights"
  | "Environmental Justice"
  | "Indigenous Rights"
  | "Disability Rights"
  | "Labor & Class"
  | "Immigration"
  | "Healthcare"
  | "Voting Rights"
  | "Education"
  | "Anti-War"
  | "Other";

export interface ArtistInfo {
  name: string;
  isAlive?: boolean;
  bio?: string;
  portraitUrl?: string;
  portraitCredit?: string;
  website?: string;
  instagram?: string;
  facebook?: string;
  tiktok?: string;
  youtube?: string;
  x?: string;
  audioUrl?: string;
  languages?: string[];
}

export interface ArtworkInfo {
  title: string;
  medium?: string;
```

```ts
  date?: string;
  location?: string;
  dateStart?: string;
  dateEnd?: string;
  city?: string;
  state?: string;
  country?: string;
  lat?: number;
  lng?: number;
  cause?: string;
  causeTags?: CauseTag[];
  imageUrl?: string;
  alt?: string;
  credit?: string;
  rights?: RightsLevel;
}

export interface MediaLink {
  title: string;
  url: string;
  source?: string;
  date?: string;
}

export interface Reflection {
  id: string;
  authorName?: string;
  text: string;
  createdAt: string;
  language?: string;
  isApproved?: boolean;
}

export interface Collection {
  id: string;
  slug: string;
  title: string;
  summary?: string;
  introMarkdown?: string;
  causeTags?: CauseTag[];
  entryIds: string[];
  featuredImageUrl?: string;
  order?: number;
}

export interface Entry {
  id: string;
  artist: ArtistInfo;
```

```
  artwork: ArtworkInfo;
  media?: MediaLink[];
  reflections?: Reflection[];
  collections?: string[];
  tags?: string[];
}

export type Artist = Entry;
```

src/lib/normalize.ts

```
import type { CauseTag, Entry, RightsLevel } from "./types";

const DEFAULT_RIGHTS: RightsLevel = "educational-ok";

const KNOWN_CAUSES: CauseTag[] = [
  "Racial Justice",
  "Women's Rights",
  "LGBTQ+ Rights",
  "Environmental Justice",
  "Indigenous Rights",
  "Disability Rights",
  "Labor & Class",
  "Immigration",
  "Healthcare",
  "Voting Rights",
  "Education",
  "Anti-War",
  "Other"
];

function coerceNumber(value: unknown): number | undefined {
  if (typeof value === "number" && Number.isFinite(value)) return value;
  if (typeof value === "string") {
    const n = Number(value);
    if (Number.isFinite(n)) return n;
  }
  return undefined;
}

function parseYear(value?: string): number | undefined {
  if (!value) return undefined;
  const text = value.trim();
  if (text.length < 4) return undefined;

  const isoLike = text.slice(0, 4);
```

```
  if ([...isoLike].every(ch => ch >= "0" && ch <= "9")) {
    const y = Number(isoLike);
    if (y >= 1800 && y <= 2100) return y;
  }

  const match = text.match(/[0-9]{4}/);
  if (match) {
    const y = Number(match[0]);
    if (y >= 1800 && y <= 2100) return y;
  }

  return undefined;
}

export function getYearRange(entry: Entry): { startYear?: number; endYear?:
number } {
  const { date, dateStart, dateEnd } = entry.artwork ?? {};
  const start = parseYear(dateStart) ?? parseYear(date);
  const end = parseYear(dateEnd);
  if (start && end && end < start) return { startYear: end, endYear: start };
  return { startYear: start, endYear: end ?? start };
}

export function getDecade(year?: number): number | undefined {
  if (!year) return undefined;
  return Math.floor(year / 10) * 10;
}

function mapLegacyCauseToTag(legacy?: string): CauseTag | undefined {
  if (!legacy) return undefined;
  const trimmed = legacy.trim();
  if (!trimmed) return undefined;

  const direct = KNOWN_CAUSES.find(c => c.toLowerCase() ===
trimmed.toLowerCase());
  if (direct) return direct;

  const lc = trimmed.toLowerCase();
  if (lc.includes("black lives") || lc === "blm") return "Racial Justice";
  if (lc.includes("women") || lc.includes("femin")) return "Women's Rights";
  if (lc.includes("lgbt")) return "LGBTQ+ Rights";
  if (lc.includes("climate") || lc.includes("environment")) return
"Environmental Justice";
  if (lc.includes("indigenous") || lc.includes("native")) return "Indigenous
Rights";
  if (lc.includes("disab")) return "Disability Rights";
  if (lc.includes("labor") || lc.includes("union") || lc.includes("worker"))
return "Labor & Class";
```

```
  if (lc.includes("immig")) return "Immigration";
  if (lc.includes("health")) return "Healthcare";
  if (lc.includes("vote") || lc.includes("election")) return "Voting Rights";
  if (lc.includes("education") || lc.includes("school")) return "Education";
  if (lc.includes("war") || lc.includes("peace")) return "Anti-War";

  return "Other";
}

export function normalizeCauseTags(entry: Entry): CauseTag[] {
  const legacyTag = mapLegacyCauseToTag(entry.artwork.cause);
  const tags = entry.artwork.causeTags ?? [];
  const merged = new Set<CauseTag>();
  for (const t of tags) merged.add(t);
  if (legacyTag) merged.add(legacyTag);
  return Array.from(merged);
}

function inferCityStateFromLocation(location?: string): { city?: string;
state?: string } {
  if (!location) return {};
  const parts = location.split(",").map(p => p.trim()).filter(Boolean);
  if (parts.length >= 2) return { city: parts[0], state: parts[1] };
  return {};
}

export function normalizeEntry(entry: Entry): Entry {
  const normalized: Entry = {
    ...entry,
    artist: {
      isAlive: entry.artist.isAlive ?? true,
      ...entry.artist
    },
    artwork: {
      rights: entry.artwork.rights ?? DEFAULT_RIGHTS,
      ...entry.artwork
    }
  };

  normalized.artwork.lat = coerceNumber(normalized.artwork.lat);
  normalized.artwork.lng = coerceNumber(normalized.artwork.lng);

  normalized.artwork.causeTags = normalizeCauseTags(normalized);

  if (!normalized.artwork.city || !normalized.artwork.state) {
    const inferred = inferCityStateFromLocation(normalized.artwork.location);
    normalized.artwork.city = normalized.artwork.city ?? inferred.city;
    normalized.artwork.state = normalized.artwork.state ?? inferred.state;
```

```
  }

  return normalized;
}

export function normalizeEntries(entries: Entry[]): Entry[] {
  return entries.map(normalizeEntry);
}
```

src/lib/filters.ts

```
import type { CauseTag, Entry } from "./types";
import { getDecade, getYearRange, normalizeEntries } from "./normalize";

export interface FilterState {
  query?: string;
  causeTags?: CauseTag[];
  mediums?: string[];
  decades?: number[];
  city?: string;
  state?: string;
  hasCoordinates?: boolean;
}

function includesCI(hay?: string, needle?: string) {
  if (!hay || !needle) return false;
  return hay.toLowerCase().includes(needle.toLowerCase());
}

export function buildAvailableCauses(entries: Entry[]): CauseTag[] {
  const set = new Set<CauseTag>();
  for (const e of normalizeEntries(entries)) {
    for (const c of e.artwork.causeTags ?? []) set.add(c);
  }
  return Array.from(set).sort();
}

export function buildAvailableMediums(entries: Entry[]): string[] {
  const set = new Set<string>();
  for (const e of entries) {
    const m = e.artwork.medium?.trim();
    if (m) set.add(m);
  }
  return Array.from(set).sort();
}
```

```
export function buildAvailableDecades(entries: Entry[]): number[] {
  const set = new Set<number>();
  for (const e of entries) {
    const { startYear } = getYearRange(e);
    const decade = getDecade(startYear);
    if (decade) set.add(decade);
  }
  return Array.from(set).sort((a, b) => a - b);
}

export function filterEntries(entriesRaw: Entry[], filters: FilterState):
Entry[] {
  const entries = normalizeEntries(entriesRaw);
  const { query, causeTags, mediums, decades, city, state, hasCoordinates } =
filters;

  return entries.filter(e => {
    const artistName = e.artist.name;
    const title = e.artwork.title;
    const medium = e.artwork.medium ?? "";
    const location = e.artwork.location ?? "";
    const cityValue = e.artwork.city ?? "";
    const stateValue = e.artwork.state ?? "";
    const tags = e.artwork.causeTags ?? [];

    if (query) {
      const q = query.trim();
      const match =
        includesCI(artistName, q) ||
        includesCI(title, q) ||
        includesCI(medium, q) ||
        includesCI(location, q) ||
        includesCI(cityValue, q) ||
        includesCI(stateValue, q);
      if (!match) return false;
    }

    if (causeTags?.length) {
      const hasAny = causeTags.some(t => tags.includes(t));
      if (!hasAny) return false;
    }

    if (mediums?.length) {
      const hasAny = mediums.some(m => includesCI(medium, m));
      if (!hasAny) return false;
    }

    if (decades?.length) {
```

```
      const { startYear } = getYearRange(e);
      const dec = getDecade(startYear);
      if (!dec || !decades.includes(dec)) return false;
    }

    if (city && !includesCI(cityValue || location, city)) return false;
    if (state && !includesCI(stateValue || location, state)) return false;

    if (hasCoordinates) {
      if (typeof e.artwork.lat !== "number" || typeof e.artwork.lng !==
"number") {
        return false;
      }
    }

    return true;
  });
}
```

src/lib/synonyms.ts

```
export const CAUSE_SYNONYMS: Record<string, string[]> = {
  "Racial Justice": ["BLM", "Black Lives Matter", "police reform"],
  "LGBTQ+ Rights": ["LGBTQ", "LGBTQIA", "queer rights"],
  "Women's Rights": ["women's rights", "feminism", "gender equality"],
  "Environmental Justice": ["climate", "climate justice", "environment"],
  "Voting Rights": ["elections", "ballot access"],
  "Labor & Class": ["workers", "union", "labor rights"]
};
```

Timeline sort drop-in

```
import { getYearRange } from "@/lib/normalize";

const sorted = [...artists].sort((a, b) => {
  const ay = getYearRange(a).startYear ?? 9999;
  const by = getYearRange(b).startYear ?? 9999;
  return ay - by;
});
```

Recipe

```
npm install
npm run dev
npm run build
npm run start
```